Date: 20

Executive Summary

Critical finding: The visualization engine is entirely text-prompt driven. Every structural guarantee the product promises — spatial preservation, window integrity, furniture identity, style control — is enforced only through advisory text instructions to a generative model. There are no masks, no segmentation, no depth conditioning, no candidates, and no output validation. The system behaves more like scene generation than constrained renovation transformation.

Six primary failure categories were identified through code analysis, each traced to specific root causes in the pipeline:

Spatial preservation	Soft text constraints only; no structural anchor	Critical
Window/background handling	Constraint covers count only, not position/content	Critical
Style instability	Style name is text; imageUrl is unused dead code	High
Moodboard weighting	3 discrete text states; images ordered after prompt	High
Furniture deformation	No region lock; furniture placed last in parts array	Medium
Error handling	No retry, no timeout, no output validation, generic 500	Medium

A. Current System Architecture

Stack

- Runtime: Node.js / TypeScript, Fastify v5
- AI model: Google Gemini 2.5 Flash Image via @google/genai v1.30.0
- Validation: Zod v4
- Transport: multipart/form-data
- Deployment: Google Cloud Run

Full Request Pipeline

```
POST /generate-visualization (multipart/form-data) ■ ▼ src/index.ts — Fastify + multipart (10 MB files, 15 files max) ■ ▼ src/controllers/main.ts :: generateVisualizationController() ■ • Guards multipart content-type ■ • Calls processVisualizationFormData() ■ • Calls generateVisualization() ■ • Returns { image: base64, metadata } ■ ■■■ src/utils/formdata.utils.ts :: processVisualizationFormData() ■ • Iterates & buffers all multipart parts into memory ■ • Validates roomImage presence & MIME type ■ • Validates moodBoardImages count (max 10) ■ • Parses styleInfluence (number), isRefinement (bool), stylePreset (JSON) ■ • Runs Zod schema validation ■ ■■■ src/prompts/visualization.prompt.ts ■ • buildInfluencePrompt() → one of 3 discrete text strings ■ • buildFurniturePrompt() → text block or empty string ■ • buildVisualizationPrompt() → template string substitution ■ ■■■ src/services/geminiService.ts :: generateVisualization() • Assembles multimodal parts array • Calls ai.models.generateContent() — ONE call, ONE candidate • Returns candidates[0].content.parts[0].inlineData.data (base64)
```

Data Shape: What Is Preserved vs Lost

FormData in	All fields + files	All buffered	Nothing yet
After Zod	Same + validated types	All	None
Prompt build	stylePreset.name, roomType, styleInfluence, hasFurniture	Text only	stylePreset.imageUrl — PERMANENTLY DISCARDED
Gemini request	roomImage, previousResultImage?, textPrompt, moodBoardImages[], furnitureImage?	Images + text	stylePreset.imageUrl (fetched nowhere)
Response	One candidate, one image part	base64 string	All other candidates (none generated)

B. Where the Pipeline Is Losing Control

The entire engine is text-prompt driven. There are zero non-text control mechanisms in production. Every structural guarantee depends on asking a generative model to comply via natural language.

img2img strength / denoise control	No	Image sent as reference; blend strength not adjustable
Inpainting / masking	No	No region preservation of any kind
Segmentation	No	Windows, walls, floors not segmented
Depth conditioning	No	
Edge conditioning	No	
ControlNet equivalents	No	
Negative prompts	No	No 'avoid' list of any kind
Seed control	No	No reproducibility; every run is random
Multiple candidates + reranking	No	Exactly one generation per request
Output spatial validation	No	Bad outputs returned identically to good ones
Retry on failure	No	
Timeout on generation call	No	Hung call hangs the request indefinitely

C. Root-Cause Analysis by Failure Mode

Failure A — Spatial Preservation

Symptom: Outputs show changed room geometry, shifted camera angle, different wall/window layout — behaves like scene regeneration rather than controlled transformation.

Technical Cause

The only spatial constraints are advisory text instructions:

```
"Do NOT change the room's core structure, such as walls, windows, or doors position."
"Maintain the original room's geometry and proportions." "Mantain the picture
perspective"
```

Gemini 2.5 Flash Image is a generative model trained to produce plausible images, not to structurally preserve input geometry. When a style request strongly implies a different spatial aesthetic, the model's generative prior overrides soft text constraints. There is no depth map, no edge lock, no structural anchor.

Code Evidence

- src/prompts/visualization.constants.ts — lines 22–27 (both prompt templates)
- src/services/geminiService.ts — no preprocessing, no mask, no control image

Fix Location

- src/services/geminiService.ts — add control structures (Tier 3)
- src/prompts/visualization.constants.ts — harden constraint language (Tier 1)

Risk / Effort / Impact

Prompt hardening: Low effort, partial improvement on conservative styles. Real fix requires structural conditioning (Tier 3, high effort, high impact).

Failure B — Window / Background Handling

Symptom: Background visible through windows is replaced; windows are obstructed; window shape and position drift between input and output.

Technical Cause

The window constraint in both prompt templates is:

```
"Do NOT change the room's core structure, such as walls, windows, or doors position."
"Mantain the amount of windows."
```

This only protects window *count*. It says nothing about position, shape, scale, or the view through the window. There is no masking or segmentation of window regions. The model has full freedom to alter what is seen through windows, change their shape, or obstruct them with design elements.

Code Evidence

- src/prompts/visualization.constants.ts — window constraint is weaker than geometry constraint
- No window segmentation, mask, or region-lock anywhere in the codebase

Fix Location

- src/prompts/visualization.constants.ts — immediate prompt fix (Tier 1)
- New preprocessing stage — window segmentation mask (Tier 3)

> **Recommended replacement constraint (Tier 1):**
> "Windows MUST remain in their exact original position, shape, and size. The view through each window — the background, light, and exterior — must be completely preserved. Do NOT obstruct windows with furniture or decor. Do NOT alter what is visible through windows."

Failure C — Style Instability

Symptom: Some styles produce reliable spatial results; others override all constraints and produce scene regeneration.

Technical Cause

A style preset is defined as:

```
export const stylePresetSchema = z.object({ name: z.string().min(1, ...), imageUrl:
z.string().url(...), // validated but NEVER USED });
```

The style name is passed as the text token {{STYLE_NAME}} in the prompt. The imageUrl is validated but **never fetched, never sent to Gemini, and never used anywhere in the pipeline.** All styles receive identical treatment in code — the same prompt template, the same constraint blocks, the same generation parameters.

Aggressive styles (whose Gemini training representation implies dramatic spatial transformations) simply outcompete soft text constraints. The model's learned prior for 'Industrial Loft' includes spatial reorganization — and there is nothing in the system to override it.

Code Evidence

- src/services/geminiService.ts line 64 — only stylePreset.name is used
- stylePreset.imageUrl is validated in schema but referenced nowhere in the service layer
- No style-specific constraint variation anywhere in the codebase

Fix Location

- src/services/geminiService.ts — fetch imageUrl and include as reference image (Tier 1)
- New src/config/styles.config.ts — style tier definitions (Tier 1–2)
- src/prompts/visualization.prompt.ts — inject tier-specific constraint blocks (Tier 2)

Failure D — Moodboard Weighting Inconsistency

Symptom: Moodboards work well with neutral styles; they appear ignored in aggressive styles. The slider feels non-functional.

Technical Cause 1 — Only 3 Discrete States

The styleInfluence slider is 0–100 continuous, but buildInfluencePrompt() collapses it to three text strings:

```
if (styleInfluence < 33) → 'Heavily prioritize the preset style...' else if (styleInfluence > 66) → 'Heavily prioritize the provided mood board images...' else → 'Blend the preset style and mood board images evenly.'
```

A slider at 1 and a slider at 32 produce identical prompts. A slider at 67 and 99 produce identical prompts.

Technical Cause 2 — Parts Ordering

In geminiService.ts, mood board images are pushed to the parts array **after** the text prompt:

```
parts.push({ text: fullPrompt }); // text FIRST parts.push(...moodBoardParts); // mood board images AFTER text
```

For multimodal models, reference images placed after an instruction may receive less contextual weight than images placed before it.

Technical Cause 3 — Style Dominance Is Emergent

When an aggressive style name carries strong generative weight in Gemini's training data, the text instruction to blend with a moodboard loses against the model's learned style representation. There is no mechanism to force moodboard influence at the model level.

Fix Location

- src/services/geminiService.ts — reorder: moodboard images before text prompt (Tier 1)
- src/prompts/visualization.prompt.ts — continuous weighting language (Tier 2)

Failure E — Furniture Deformation

Symptom: Uploaded furniture is usually included but gets structurally deformed, especially headboards. Object identity is not reliably preserved.

Technical Cause

Furniture is passed as a raw image with text instructions to 'incorporate this exact piece of furniture.' There is no region locking, no segmentation of the furniture from its background in the upload, and no mechanism to force the model to treat the furniture as an immutable object. When an aggressive style is active, Gemini's style transformation impulse deforms the furniture to match the style aesthetic.

Additionally, the furnitureImage is pushed **last** in the parts array, after all mood board images — reducing its positional influence relative to the room image and prompt.

Code Evidence

- src/services/geminiService.ts lines 86–89 — furniture pushed last
- src/prompts/visualization.constants.ts lines 65–82 — furniture constraint is advisory text only

Fix Location

- src/services/geminiService.ts — move furniture before text prompt (Tier 1)
- src/prompts/visualization.constants.ts — add structural identity preservation language (Tier 1)

Failure F — Error Handling and Resilience

Symptom: Upload failures return generic 500; generation failures are silent; no retry; no fallback; no output quality gate.

Technical Cause 1 — No Timeout

```
// src/services/geminiService.ts const response = await ai.models.generateContent({ ... }); // No timeout wrapper — a hung call hangs the request indefinitely
```

Technical Cause 2 — Generic 500 Catch

```
// src/controllers/main.ts console.log(error) // debug artifact still in production ... return reply.status(500).send({ error: 'Internal Server Error', message: 'Ocurrido un error al procesar la solicitud', });
```

The actual Gemini error (content blocked, rate limit, model error) is logged only. The debug console.log() before the ValidationError check is a leftover artifact.

Technical Cause 3 — No Output Validation

```
if (firstPart && firstPart.inlineData && firstPart.inlineData.data) { return firstPart.inlineData.data; // no quality check } else { throw new Error('No image was generated...'); }
```

Bad outputs — wrong geometry, broken windows, deformed furniture — are returned to the user identically to good outputs. There is no quality threshold.

Additional Missing Mechanisms

- No retry logic — one Gemini failure fails the whole request
- No multiple candidate generation — single shot with no safety net
- No content validation on uploads — a valid JPEG of a dog passes all checks

Fix Location

- src/services/geminiService.ts — add Promise.race() timeout + retry (Tier 1)
- src/controllers/main.ts — remove console.log, surface error type to client (Tier 1)
- src/services/geminiService.ts — add basic output validation (Tier 2)

D. Style System Assessment

Current state: Style is a name string. The style reference image URL is dead code. All styles are treated identically in the pipeline. There is no tier system, no risk profile, no style-specific constraint, no intensity control, and no demo-safe curation.

The Dead Code Finding — stylePreset.imageUrl

```
// stylePreset.imageUrl is validated in schema... export const stylePresetSchema =
z.object({ name: z.string().min(1, ...), imageUrl: z.string().url(...), // ← validated
}); // ...but only name survives to the generation service const fullPrompt =
buildVisualizationPrompt({ stylePresetName: stylePreset.name, // only name used //
imageUrl never referenced again });
```

What Is Missing

- Style tier (conservative / moderate / aggressive / experimental)
- Style risk flag
- Style-specific constraint overrides
- Demo-safe style curation
- Production vs experimental mode gate
- Style intensity knob separate from moodboard influence
- Style reference image sent to generation model

Where to Introduce Style Tiering

A new **src/config/styles.config.ts** should define:

```
type StyleTier = 'conservative' | 'moderate' | 'aggressive' | 'experimental'; interface
StyleConfig { name: string; tier: StyleTier; additionalConstraints: string[];
maxStyleInfluence: number; // cap slider at model level demoSafe: boolean; }
```

buildVisualizationPrompt() should accept a StyleConfig and inject tier-specific constraint blocks into the prompt. Conservative styles get no extra constraints. Aggressive styles get reinforced spatial lock and window integrity language.

E. Multi-Input Blending Assessment

Room image	First in parts array	Implicit (positional)	Adequate
Style preset name	Text string in prompt	None	Only soft text
Style preset imageUrl	NEVER SENT	N/A	Dead code — biggest missed opportunity
Moodboard images	Pushed after text prompt	3 discrete text states	Ordering hurts influence; 3 states ≠ continuous
Furniture image	Last in parts array	None	Positional de-emphasis
textPrompt	Embedded in template	None	Competes with style/moodboard
previousResultImage	Second in array (refinement)	None	Adequate position

Correct Parts Ordering

Current (broken) order: [room image] → [previousResult?] → [text prompt] → [moodboard images] → [furniture image]

Recommended order: [room image] → [style reference image] → [moodboard images] → [furniture image] → [text prompt]

There is no conflict resolution logic in the system. No rule such as 'if furniture image is present, cap style transformation intensity.' No protection against style overriding furniture identity. No priority hierarchy between inputs.

F. Validation / Fallback / Recovery Assessment

The system has none of the following: multiple candidate generation, spatial consistency check, window preservation check, furniture presence check, quality scoring, retry with adjusted parameters, fallback prompt, bad-output suppression, generation timeout, or meaningful client-facing error detail. Every generation is a single shot with no safety net.

Consequence

A bad generation — wrong geometry, broken windows, deformed furniture — is returned to the user identically to a good one. A Gemini content block returns a 500 with no explanation. A hung Gemini call hangs the HTTP connection until Fastify's connection timeout fires. The user receives no actionable feedback.

G. Recommended Fixes by Priority Tier

Tier 1 — 1–2 Sprints, Highest Impact

T1-1: Fix parts ordering in geminiService.ts

Move moodboard images and furniture image before the text prompt. This alone will improve moodboard influence measurably and reduce furniture de-emphasis.

File: src/services/geminiService.ts — reorder the parts.push() calls

T1-2: Fetch and use stylePreset.imageUrl as a reference image

The style preset has a reference image URL that is validated but never used. Fetching it and including it in the parts array before the text prompt gives the model a visual reference for the style — far more powerful than the name string alone. This is the single highest-leverage fix in the codebase.

File: src/services/geminiService.ts — add fetch(stylePreset.imageUrl) → buffer → insert before text

T1-3: Strengthen window constraint

Change from "maintain count" to: "Windows MUST remain in their exact original position, shape, and size. The view through each window must be completely preserved. Do NOT obstruct windows with furniture or decor."

File: src/prompts/visualization.constants.ts — both prompt templates

T1-4: Add timeout and retry logic

Wrap generateContent() in Promise.race() with a 30-second timeout. Add retry up to 2 attempts with exponential backoff on failure.

File: src/services/geminiService.ts

T1-5: Surface meaningful error messages

Catch Gemini error types (content block vs rate limit vs model error) and return distinct client-facing error codes. Remove the debug console.log(error) artifact.

File: src/controllers/main.ts

T1-6: Introduce style tier config and inject tier-specific constraints

Create src/config/styles.config.ts classifying styles as conservative / moderate / aggressive. For aggressive styles inject an extra constraint block reinforcing spatial preservation and window integrity. Fastest path to making style behavior unequal.

File: New src/config/styles.config.ts, src/prompts/visualization.prompt.ts

T1-7: Strengthen furniture identity prompt

Add instruction that the furniture's structural form must not be redesigned to match the room style — only placement and lighting should adapt.

File: src/prompts/visualization.constants.ts — INSTRUCTION_INTEGRATE_FURNITURE

Tier 2 — Moderate Refactoring

T2-1: True continuous moodboard weighting

Replace 3 discrete text strings with language that communicates the actual slider value. E.g., "The moodboard images should account for approximately {styleInfluence}% of the stylistic direction." More expressive than the current ternary.

File: src/prompts/visualization.prompt.ts — buildInfluencePrompt()

T2-2: Style-specific constraint injection system

Extend style config to carry per-style constraint blocks injected into both prompt templates. Conservative styles: no extra constraints. Aggressive styles: reinforced spatial lock and window integrity language.

File: src/config/styles.config.ts, src/prompts/visualization.prompt.ts

T2-3: Basic output validation

After receiving the generated base64 image, decode it and check: (a) is it a valid image? (b) do dimensions approximately match the input? If not, retry once before returning an error.

File: src/services/geminiService.ts

T2-4: Upload content validation

After MIME check, run a basic image decode to confirm the file is a real image with valid dimensions. Reject images below minimum resolution.

File: src/utils/validation.utils.ts

T2-5: Structured design spec (replace template string substitution)

Replace the {{VARIABLE}} string-substitution system with buildDesignSpec() returning a structured object, then buildPrompt(spec) serializing it. Makes prompt logic testable and extensible without string manipulation.

File: src/prompts/visualization.prompt.ts — full refactor

Tier 3 — Target State Architecture

T3-1: Window masking / segmentation pass

Before sending to Gemini: run segmentation (Cloud Vision, SAM, or dedicated preprocessing) to identify window regions. Pass window mask as separate control image or inpainting mask. This is the only way to guarantee window content preservation — text instructions cannot.

T3-2: Structural anchor conditioning

Detect walls, floor, ceiling, and structural edges using depth or edge detection. Use as control signals to guide generation within structural constraints. Requires a preprocessing service.

T3-3: Multiple candidate generation + reranking

Generate 2–3 candidates per request. Score each for: (a) spatial consistency, (b) window preservation, (c) furniture presence. Return highest-scoring candidate. Reranking can be done with a secondary Gemini vision call.

T3-4: Production vs experimental mode gate

Runtime flag VISUALIZATION_MODE=production|experimental. In production mode, only conservative and moderate tier styles are enabled. Aggressive styles require experimental mode. Gives product team control over what reaches demos and customers.

T3-5: Structured constraint compiler

Replace entire prompt-building system with a constraint hierarchy: spatial > window > furniture > style > user request. Compiler enforces lower-priority constraints cannot override higher-priority ones. Style intensity for aggressive styles is automatically capped when spatial constraints are active.

H. Target-State Architecture

```
POST /generate-visualization ■ ▼ Input Validation Layer • MIME type + content validation • Room image dimension check • Style tier lookup → demoSafe flag ■ ▼ Preprocessing Layer (NEW) • Fetch stylePreset.imageUrl → style reference image • (Future) Window segmentation mask • (Future) Structural edge map ■ ▼ Constraint Compiler (NEW) • Priority: spatial > window > furniture > style > user request • Style tier → inject tier-specific constraint block • Style influence → continuous language (not 3 states) • Production mode → cap aggressive styles ■ ▼ Multimodal Request Assembly • [room image] → [style ref image] → [moodboard images] → [furniture image] → [compiled prompt] ■ ▼ Generation Service (timeout + retry) • N candidates (2–3 in target state) • Timeout: 30s per attempt, max 2 retries ■ ▼ Output Validation Layer (NEW) • Dimension check • Basic spatial consistency check • Reranking if multiple candidates ■ ▼ Response • Best-scored image • Metadata: style tier, constraint flags applied • Meaningful error codes on failure
```

I. Open Questions / Unverified Areas

gemini-2.5-flash-image model capabilities
The model ID used is not a standard publicly documented Gemini model name. Confirm this is the correct model string and whether it supports any form of image conditioning, masks, or reference image weighting not yet explored.

stylePreset.imageUrl asset ownership
If these are hosted images, fetching them server-side at generation time needs CORS, latency, and caching consideration. Origin and caching strategy need to be defined.

Frontend style catalog
The frontend (reformai_vis_react) likely has a known list of style names. The StyleConfig tier definitions should originate there — needs cross-repo coordination.

prompt.backup.txt
This file is identical to visualization.constants.ts and appears to be a stale checkpoint. It should be deleted or versioned properly — having it in-repo as a stale reference is a maintenance liability.

Vertex AI vs public Gemini API
The Cloud Run deployment context suggests GCP. Vertex AI alternatives may offer better SLAs, additional model control parameters, or image conditioning features not available via the public API.

Test coverage
package.json scripts: "test": "echo \"Error: no test specified\" && exit 1". The entire pipeline — prompt building, influence weighting, parts assembly — is untested. Any Tier 1 fix should be accompanied by unit tests for buildInfluencePrompt(), buildVisualizationPrompt(), and buildFurniturePrompt().